3. Run your change block a second time and store the result, array.size, as the *after* value
4. Pass the expectation if the before and after values are different

This expectation checks *whether* or not the expectation changed, without regard to *how much*. For that, we'll need to turn to another technique.

Specifying Change Details

Like other RSpec fluent matchers, the change matcher offers an easy way to give details about the change. Specifically, you can use by, by_at_least, or by_at_most to specify the *amount* of the change:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
expect { array.concat([1, 2, 3]) }.to change { array.size }.by(3)
expect { array.concat([1, 2, 3]) }.to change { array.size }.by_at_least(2)
expect { array.concat([1, 2, 3]) }.to change { array.size }.by_at_most(4)
```

If you care about the exact before and after values, you can chain from and to on to your matcher (either individually or together):

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
expect { array << 4 }.to change { array.size }.from(3)
expect { array << 5 }.to change { array.size }.to(5)
expect { array << 6 }.to change { array.size }.from(5).to(6)
expect { array << 7 }.to change { array.size }.to(7).from(6)
```

It probably doesn't surprise you to hear that you can also pass a matcher (or any object that implements the === protocol) into from and to:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
x = 5
expect { x += 10 }.to change { x }
  .from(a_value_between(2, 7))
  .to(a_value_between(12, 17))
```

Note that there's a bit of a gotcha to passing a matcher, at least if you only use to or from (and not both). Consider this expectation:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
x = 5
expect { x += 1 }.to change { x }.from(a_value_between(2, 7))
```

This expectation passes, because the value of x changed, and it was originally a value between 2 and 7. However, as this reads, you might expect that it would only pass if the final value of x was no longer between 2 and 7. If you care about both the *before* and *after* values, it's a good idea to specify both from and to.